

Lazy Loading Using the Router



Lara Newsom

Software Technical Leader | Speaker | Angular GDE

[lara-newsom](#) | www.linkedin.com





Large Initial Bundle Sizes

Adding features -> larger initial bundles

Larger initial bundles -> slower page loads

Slower page loads -> High bounce rates

High bounce rates -> missed profits



Code Splitting in Angular

Code splitting is performed by esbuild at build time

Developers make choices that define code boundaries

Boundaries can be defined using standalone architecture, reusable components, and a clear separation of concerns

Routes can also define code boundaries



Code splitting alone does not reduce the size of the initial bundle.



A ginger cat is lying on its side on a red, wrinkled blanket. The cat's front paws are stretched out, and its hind legs are also extended. The cat's eyes are closed, and it appears to be sleeping or resting. The background is a solid white color.

Angular Lazy Loading

Lazy loading tells Angular, “It is ok to wait to download a chunk of code”

Code split into separate JavaScript files are called chunks

Chunks in the initial code bundle are listed in the build output section called initial chunk files

Lazy-loaded chunks are listed in the build output in the lazy chunk files section



Lazy Loading in Angular

Lazy-loaded routes

Deferrable views



Declaring Lazy Loading in Routes



Route Component Property

Used to load components into RouterOutlets

Components loaded using the component property are eagerly loaded

The component code is included in the same bundle as its parent



Route Properties for Lazy Loading

loadChildren

Used to lazy load child routes

vs.

loadComponent

Used to lazy load a component



Declare a Lazy-loaded Route

```
// loadChildren resolves to a child route declaration
{
  path: 'lazy-child-routes',
  loadChildren: () =>
    import('./path-to-lazy-child-routes')
      .then(m => m.LAZY_CHILD_ROUTES)
},
// loadComponent resolves to a component
{
  path: 'lazy-component',
  loadComponent: () =>
    import('./path-to-lazy-component')
      .then(m => m.LAZY_COMPONENT)
},
```



Lazy-loaded Routes

loadChildren

Child Routes

Parent Route

loadComponent

ChildComponent

Child RouterOutlet



Downloaded code remains in the browser until the cache is cleared or the browser is closed.



Application Build Output

Initial chunk files	Names	Raw size	Estimated transfer size
→ main-FHANT5UR.js	main	573.62 kB	137.44 kB
→ styles-0KPFCP3R.css	styles	104.11 kB	7.91 kB
	Initial total	677.73 kB	145.35 kB

Generated using the `ng build` command





Demo: Implement Lazy Loading

Use `loadChildren` and `loadComponent` to reduce the size of the initial bundle using lazy loading.



Route Preloading



Preloading can reduce the amount of time users have to wait before lazy-loaded views are rendered.





withPreloading Strategies

Used to preload lazy-loaded code chunks

NoPreloading is the default behavior; lazy chunks are not downloaded until they are requested

PreloadAllModules starts downloading all lazy chunks immediately after the initial bundle is downloaded

Small- to medium-sized apps work well with PreloadAllModules

Larger apps should fine-tune preloading with a custom strategy





Custom Preloading Strategy Example

<https://angular.dev/guide/routing/customizing-route-behavior#creating-a-custom-preloading-strategy>

